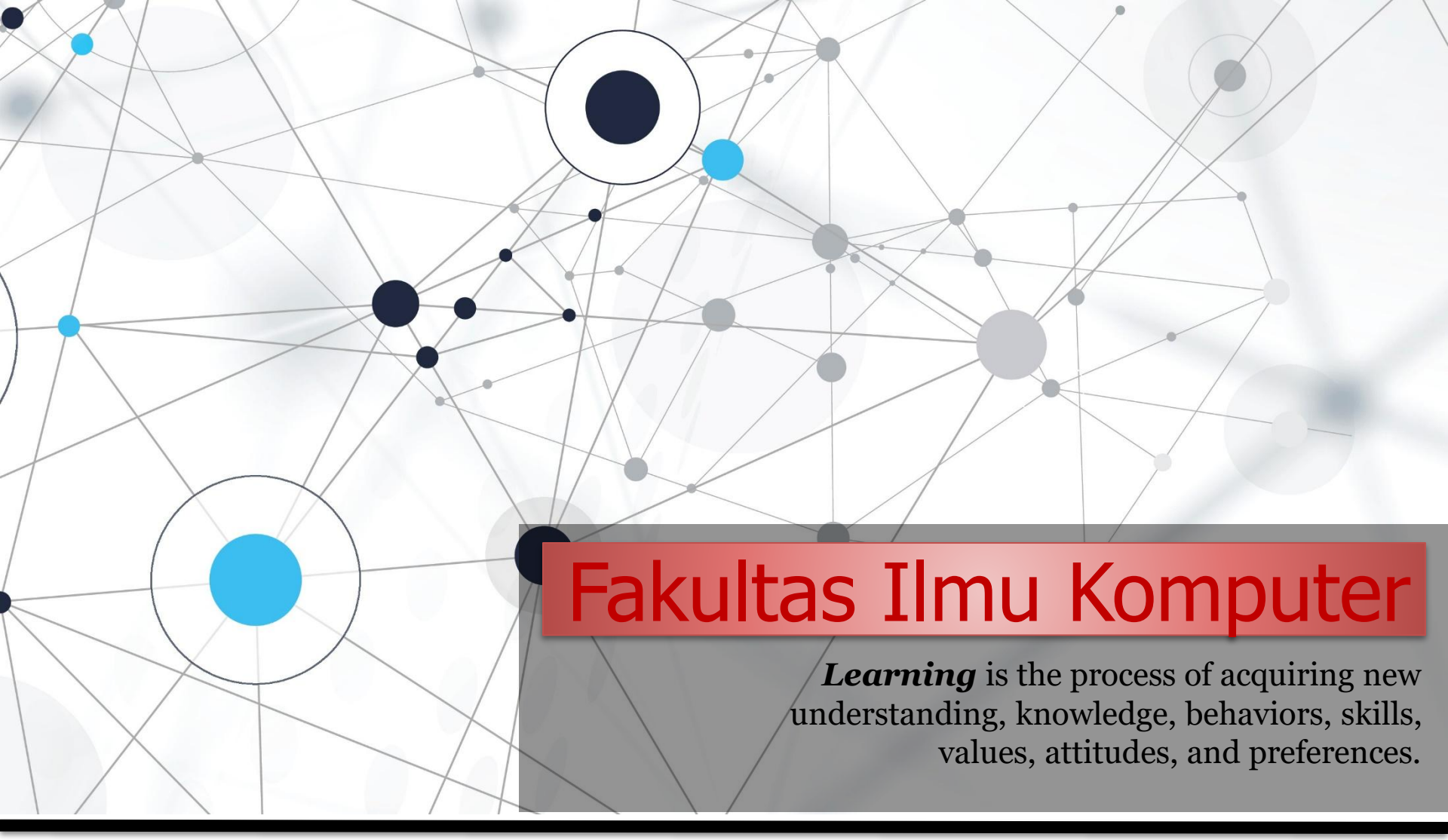
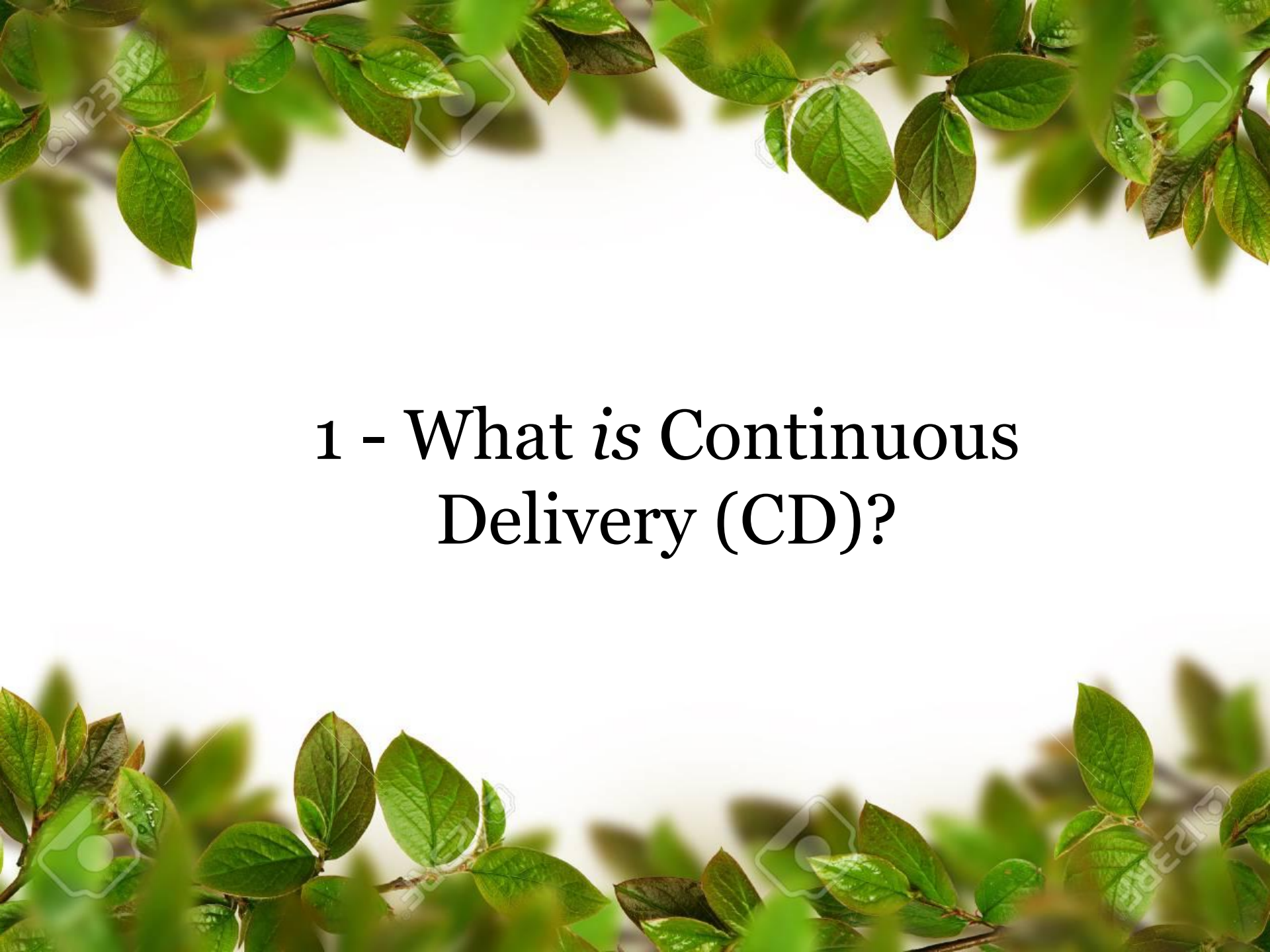


Lecture 11 & Lecture 12

Continuous Deployment (CD)

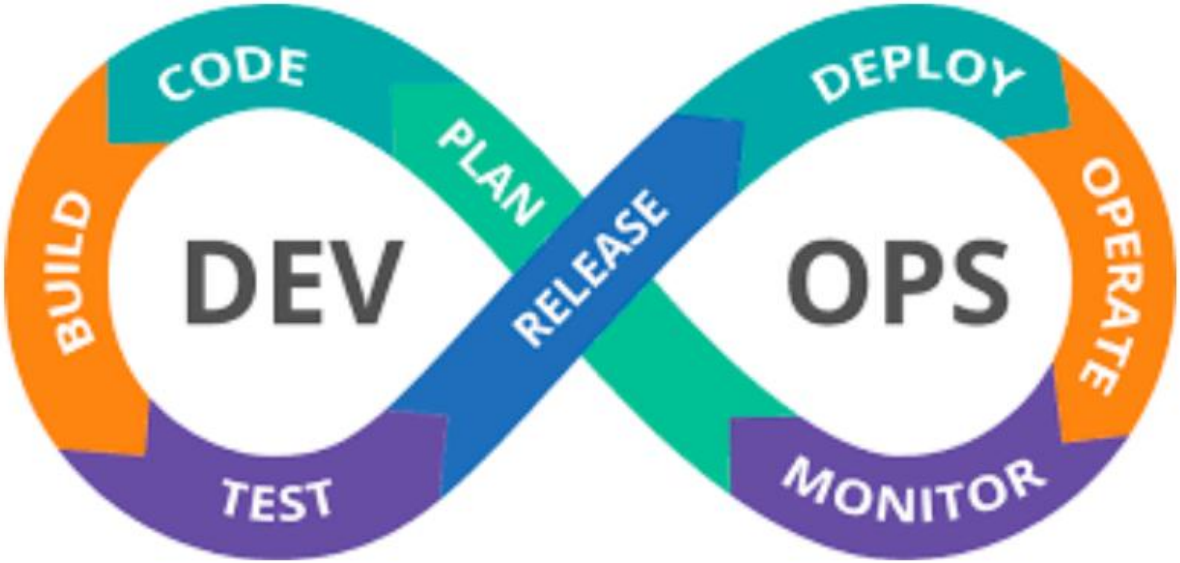
Welcome to *the* DevOps Engineering *course*



1 - What *is* Continuous Delivery (CD)?

Key Components *of* Devops

The key components of DevOps are Continuous Integration, Continuous Testing, Continuous Development, Continuous Feedback, Continuous Monitoring, Continuous Deployment and Continuous Monitoring.



Step Deploy:

The code is deployed in a production environment for usage by customers.

Continuous Deployment (CD)

- Once **continuous integration has been successfully completed**, the next step is to deploy the application automatically in one or more non-production environments, which is called staging. This process is called **continuous delivery (CD)**.

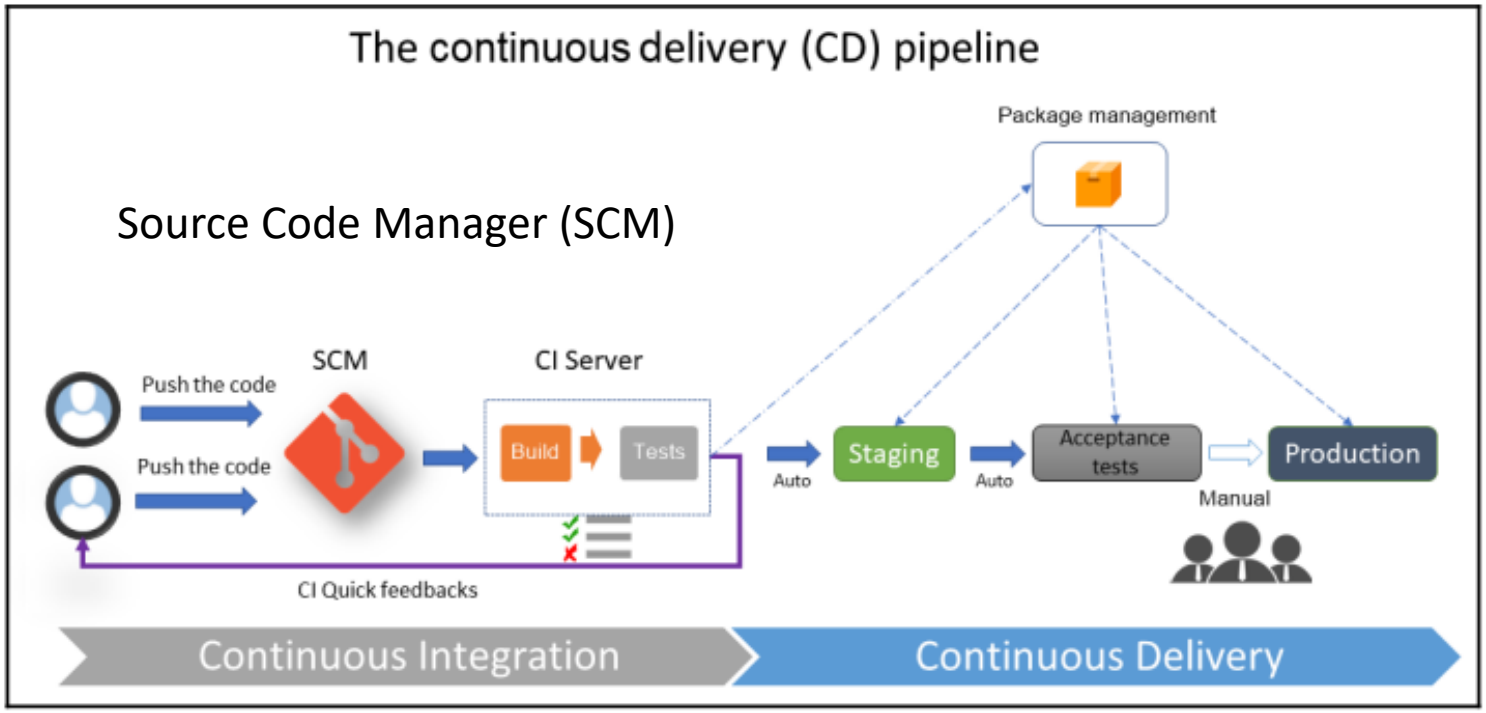
- CD often **starts with an application package prepared by CI**, which will be installed according to a list of automated tasks.

- These tasks can be of any type: **unzip**, **stop** and **restart service**, **copy files**, **replace configuration**, and so on.

- The **execution of functional** and **acceptance tests** can also be performed during the CD process.

- Unlike CI, CD aims to test the entire application with all of its dependencies. This is very visible in microservices applications composed of several services and APIs; CI will only test the microservice under development while, once deployed in a staging environment, it will be possible to test and validate the entire application as well as the APIs and microservices that it is composed of.

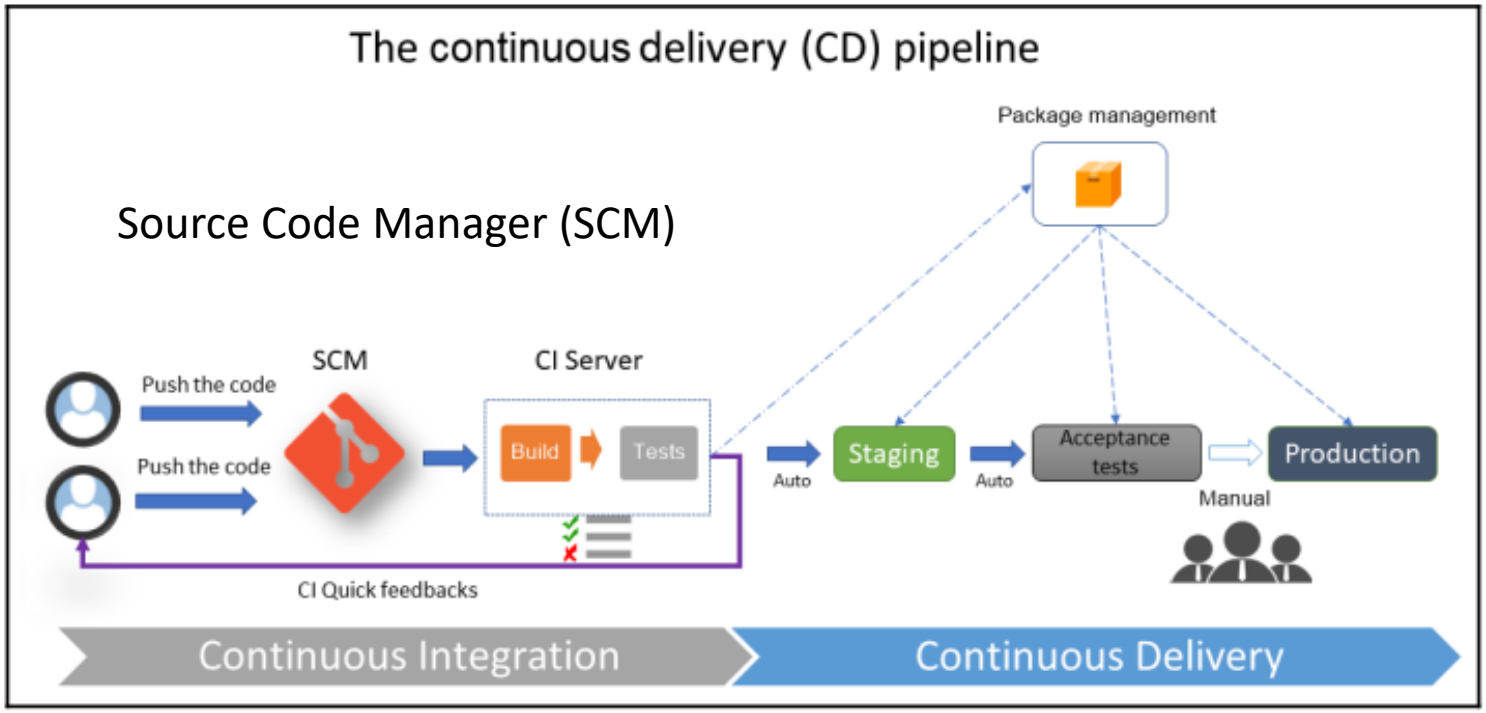
Continuous Deployment (CD)



- Continuous Delivery (CD) is one of the essential practices in the DevOps that aims **to automate and ease the process of delivering software to production** or test environments on a continuous basis.

- CD is an extension of Continuous Integration (CI), which is a practice that ensures developed code is continuously tested and integrated into the main repository quickly.

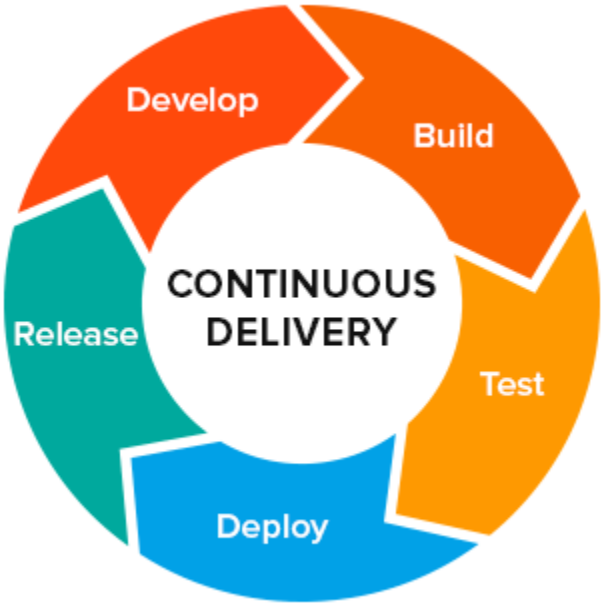
Continuous Deployment (CD)



1) What is important in a CD process is that the deployment to the production environment, that is, to the end user, is **triggered manually by approved users**.

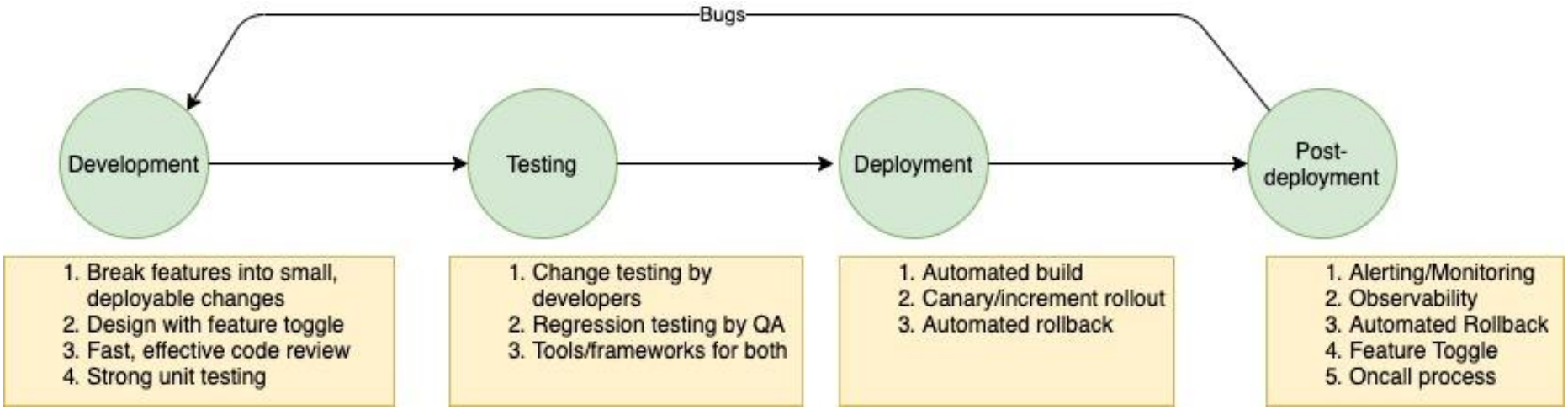
2) This diagram clearly shows that the CD process is a continuation of the CI process. It represents the chain of CD steps, which are automatic for staging environments but manual for production deployments. It also shows that the package is generated by CI and is stored in a package manager and that it is the same package that is deployed in different environments.

Continuous Deployment (CD)



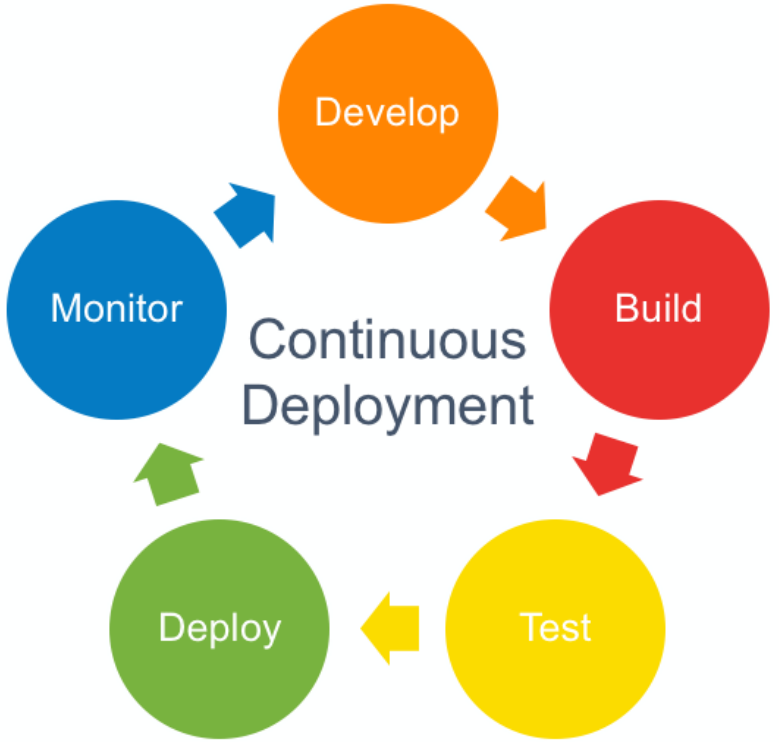
Continuous Delivery is a software development process that helps developers release software reliably (*andal/dapat dipercaya*) and quickly (*cepat*).

Continuous Deployment (CD)

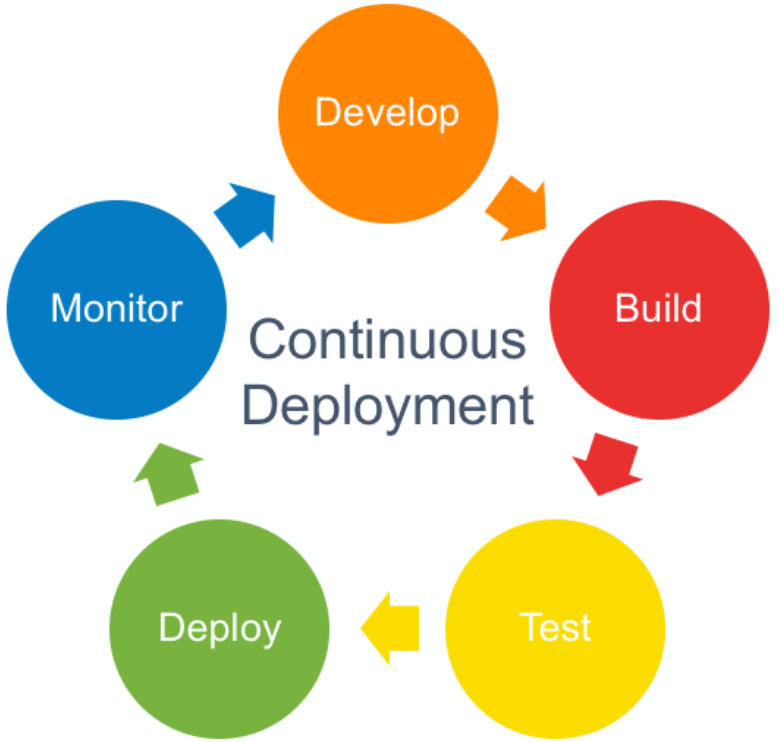


The goal of Continuous Delivery (CI) is to enable the quick and reliable delivery of new features, updates, and fixes to users. This can be done by **automating the entire software development process** from start to finish.

Continuous Deployment (CD) is a software development process that allows for the automatic **deployment of new code versions to a software system**, typically as part of an **automated build and release process**.



CD is important because it allows developers to **quickly respond to changes in the codebase by automatically deploying the latest version without having to wait** for a manual approval process or waiting for someone else to deploy the code.



Continuous Deployment (CD)

Continuous Deployment (CD) helps to improve quality control and keeps the software system up-to-date with the latest changes in the codebase.

It **reduces the time** it takes to release new code.

It **reduces the amount of work** that needs to be done after a code release has been made.

It **improves communication** between developers and QA personnel.

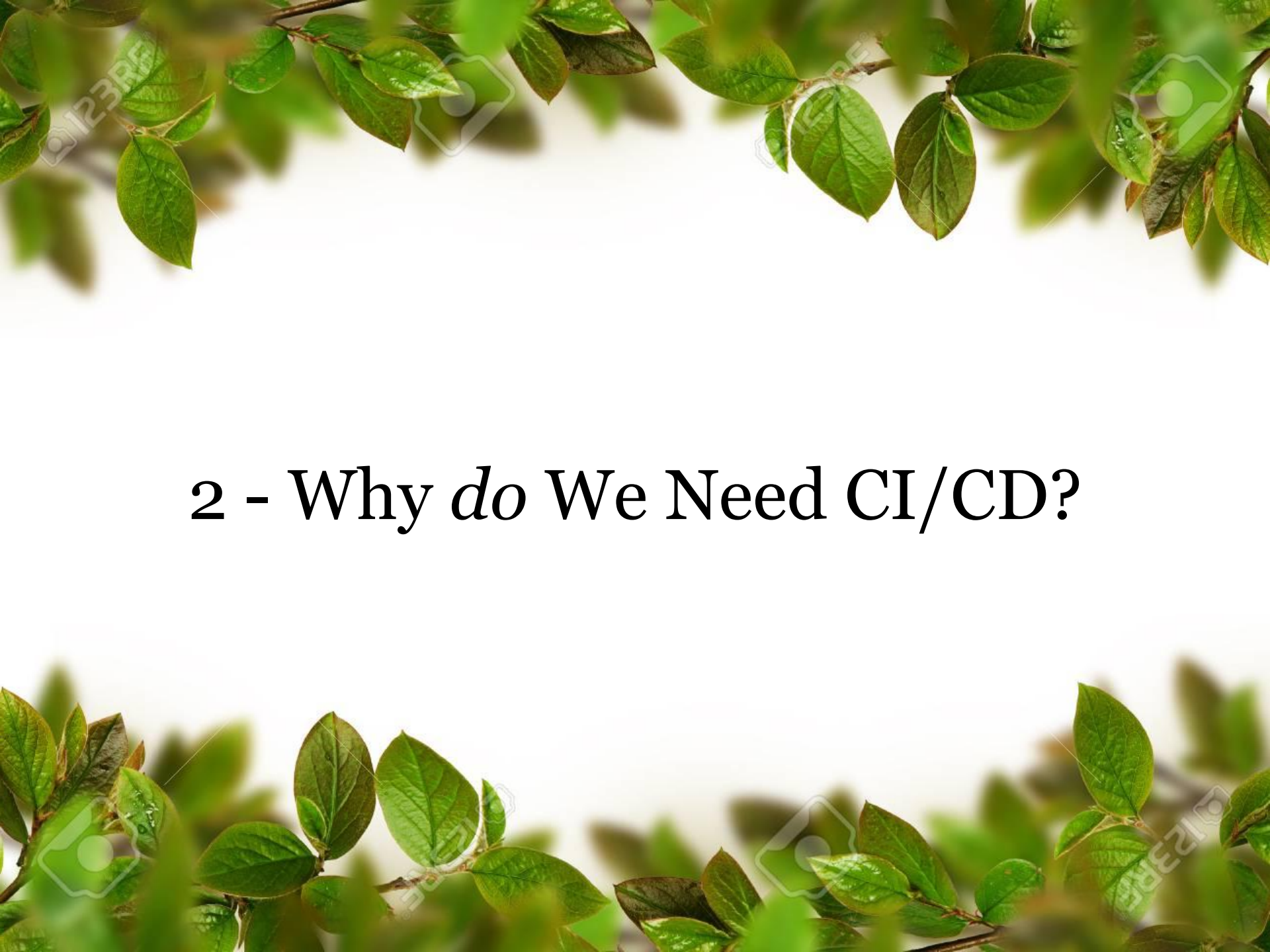
It makes it easier to **track changes** and rollbacks.

CI *vs* CD *vs* CD

1) **Continuous Integration (CI)** is a **software development process** that helps developers integrate changes to code into the main branch of the project as often as possible.This helps avoid potential integration issues and makes it easier to track changes.

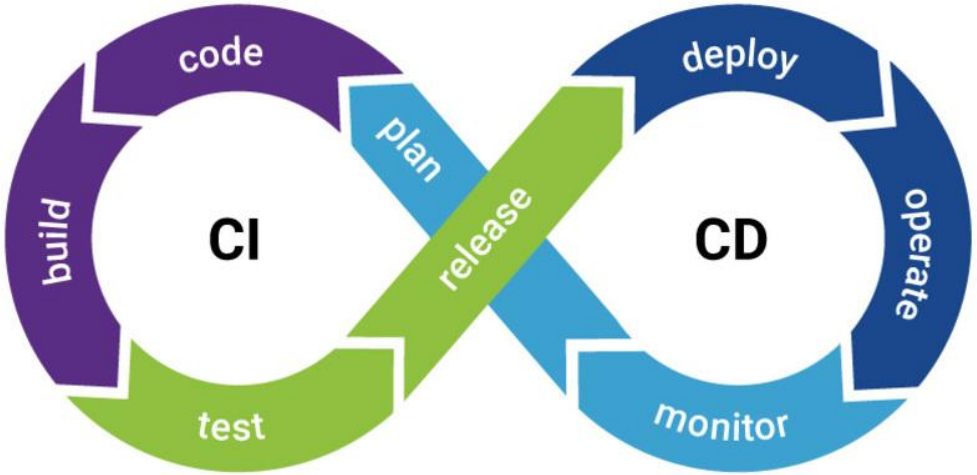
2) **Continuous Delivery (CD)** is a **software delivery process** that ensures that **all new features, fixes, and updates are released in a quality-controlled** manner. This helps minimize the risk of defects entering the live system.

3) **Continuous Deployment (CD)** is a **software deployment process** that automates the deployment of new versions of applications to designated servers. This helps ensure that the application is always available and **meets customer expectations**.



2 - Why *do* We Need CI/CD?

Benefits *of* CI/CD pipeline



Improved Quality Control
CD helps to ensure that all changes are released in a controlled manner, leading to improved quality. This can help to avoid the introduction of defects into live systems.

Reduced Risk
Automated deployment ensures that applications are always available and meet customer expectations. This helps to reduce the risk of introducing defects into live systems.

Speed to market
Using a CI/CD pipeline can help you improve your speed to market by releasing new versions of your software more often.

Challenges *of* CI/CD pipeline

The challenges of using a CI/CD pipeline include:

Time-consuming
CD can be time-consuming due to the need for manual testing and deployment.

Lack of control
Some changes may not be accepted by the development team, leading to integration issues.

Performance issues
CD can cause performance problems due to the increased load on the development and testing systems.

Why *do* we need CI/CD?

In DevOps, getting **started with a CI/CD pipeline requires collaboration** between development, IT, and operations teams on CI/CD technologies, best practices, and priorities. Once these groups are aligned, they can start reaping the full benefits of CI/CD by implementing an automated software release pipeline that ensures quality, security, and stability, among other things.

[1] Tingkat Rilis Lebih Cepat (Faster Release Rate)

DevOps is about accelerating the release rate of new software versions. A well-functioning CI/CD pipeline can help achieve this by automating the deployment process and reducing the amount of manual work required. This speeds up the feedback loop, allows for better testing and validation, and ultimately results in a more stable product.

[2] Mengurangi Cacat Kualitas (Reduced Quality Defects)

One of the main benefits of using a CI/CD pipeline is that it helps reduce quality defects.Automated deployments and rigorous testing minimize the chance of human error, meaning that new versions are released with fewer errors. This leads to higher customer satisfaction and increased ROI for the software development team.

[3] Mengurangi Biaya (Reduced Costs)

CI/CD pipeline can also save your team money by reducing the amount of time it takes to deploy new versions of software.By automating the process, we can reduce or eliminate the need for human involvement in the deployment process and reduce operational overhead.

CI/CD Tools & Platforms

Jenkins
Jenkins is one of the most commonly used CI/CD tools out there, as it's easy to use and has a wide range of integrations.

Bamboo
Bamboo is a popular CI/CD tool that's widely used in the enterprise world. It has a lot of features, but can be difficult to learn and use.

Travis CI
Travis CI is a popular open-source CI/CD tool that's used by a variety of organizations, including Microsoft and Netflix. It has easy to use build automation features and supports a wide range of languages and platforms.

GIT
Git is the most popular version control system (VCS) and is widely used in the development world. It's perfect for use in a CI/CD pipeline, as it has built-in functionality for automating deployments. Additionally, Git can be used to track changes made to software code and track revertions.

Exercise *for* Students

[1]

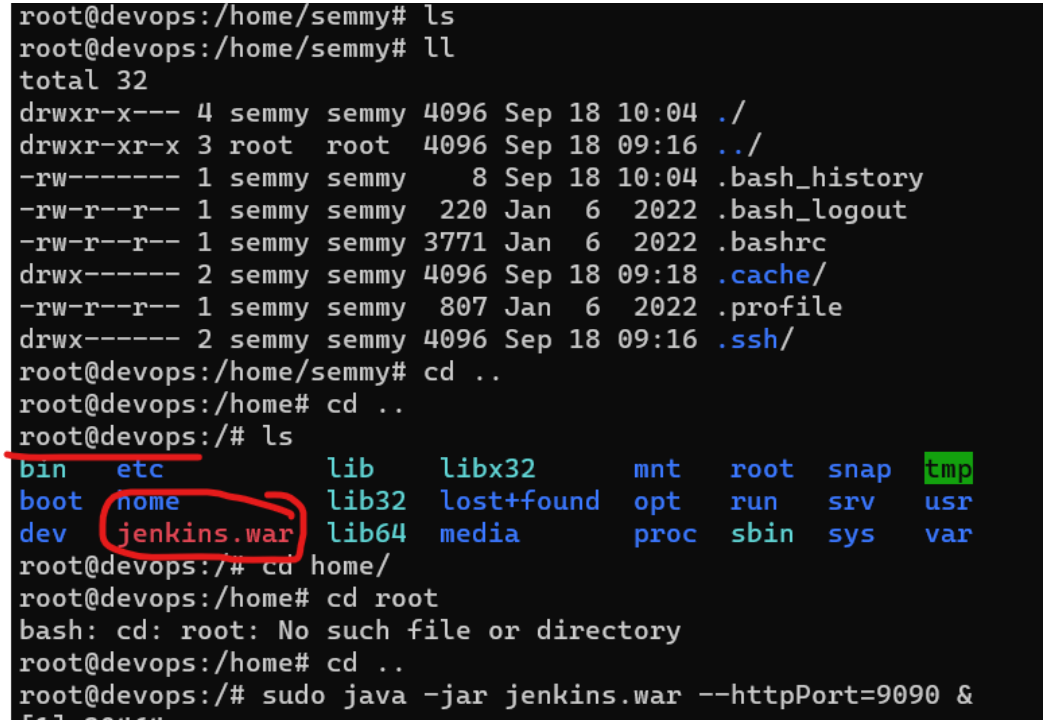
Introduction *to* basic Jenkins

Jenkins

[1] Start Jenkins

Saya install di *root directory*, jadi saat restart harus masuk ke directory yang sama. Selanjutnya type:

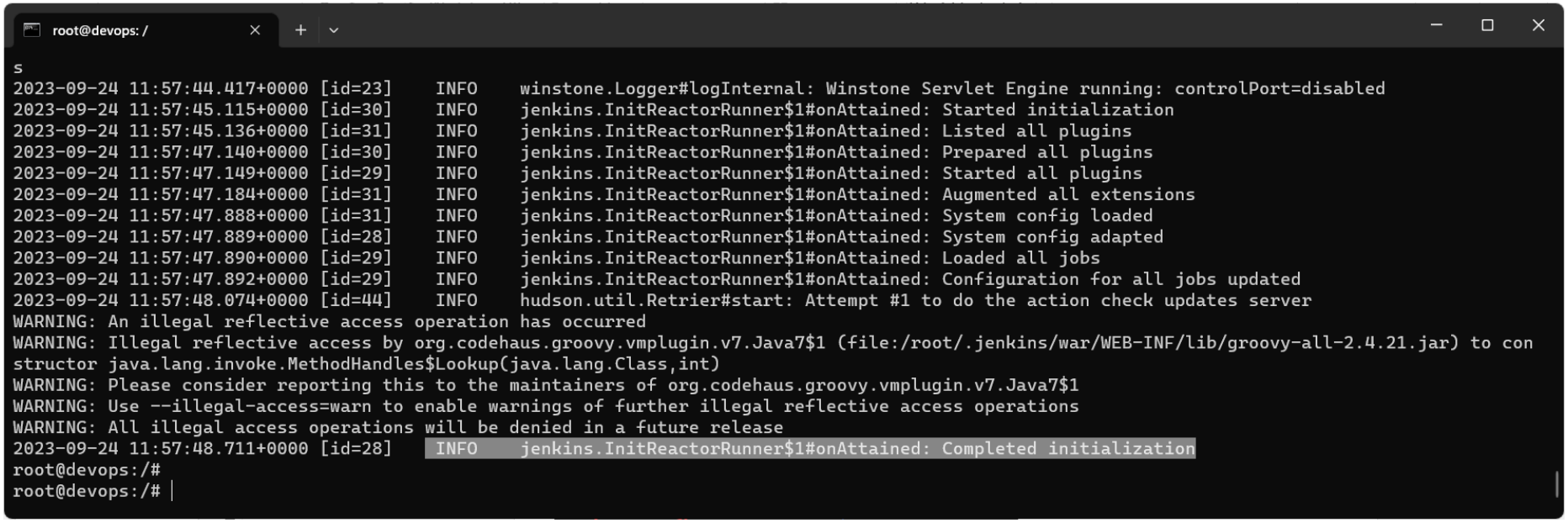
- **sudo su**
- **sudo java -jar jenkins.war --httpPort=9090 &**



Akan Muncul Error jika directory tidak sesuai:

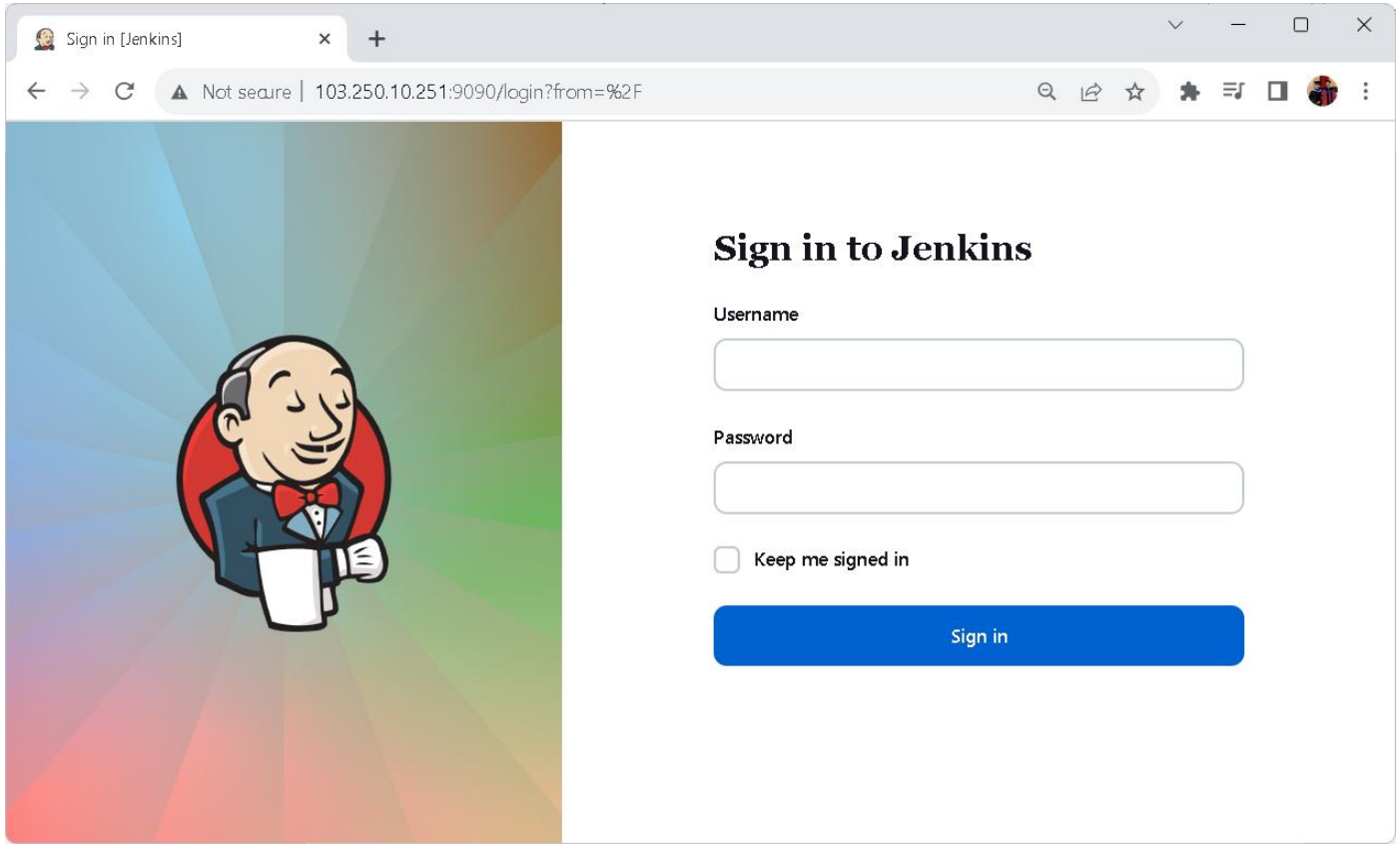
- *Error: Unable to access jarfile jenkins.war*

Jenkins Completed Initialization



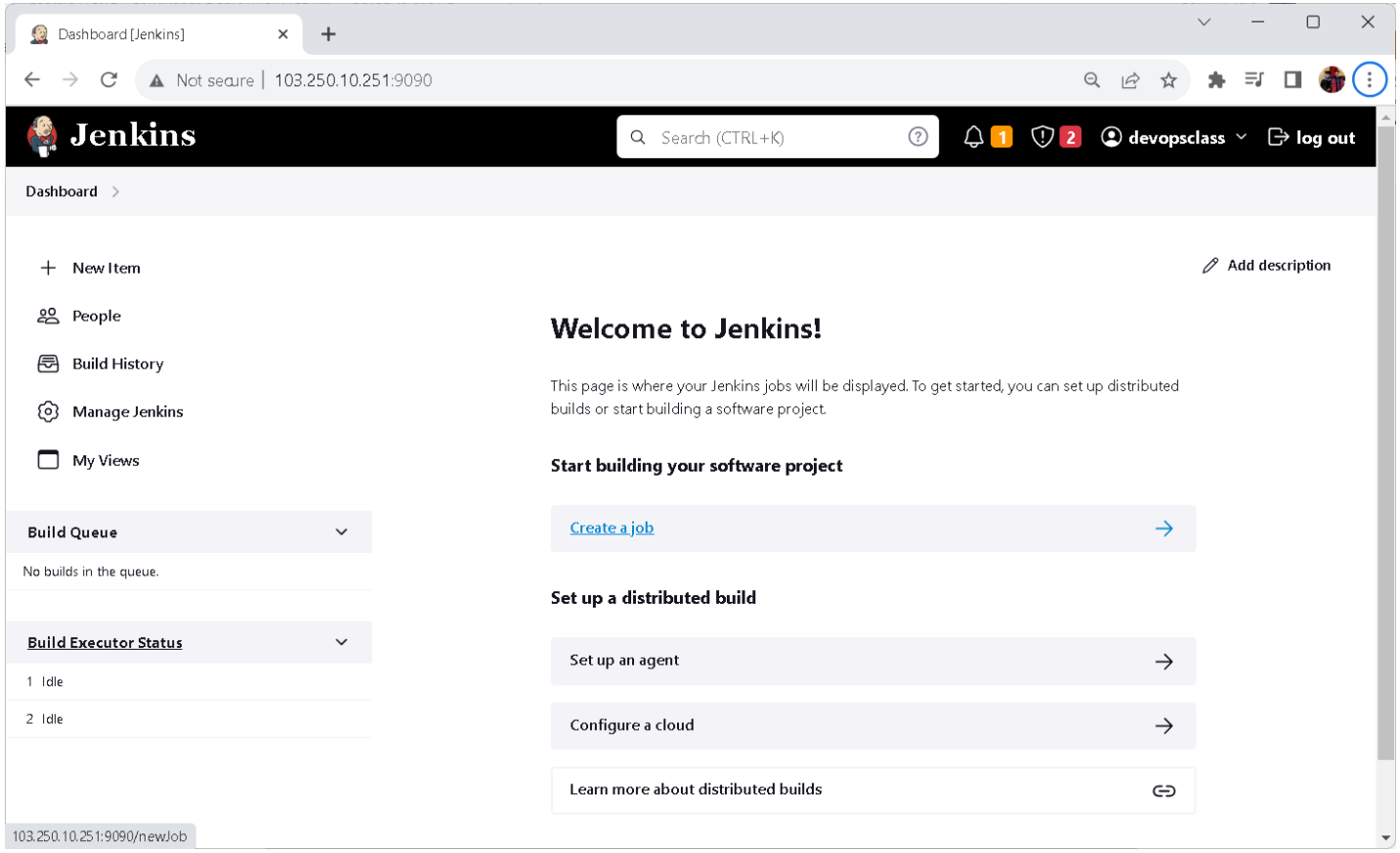
[2] Sign *Into* Jenkins 2023

Jenkins Default URL:
http://103.250.10.251:9090/



[3] Dashboard Jenkins 2.414.1

Jenkins adalah salah satu alat yang sangat populer dalam ekosistem DevOps yang digunakan untuk otomatisasi berbagai tahap dari siklus pengembangan perangkat lunak, termasuk integrasi terus-menerus (Continuous Integration, CI) dan pengiriman terus-menerus (Continuous Delivery, CD). Jenkins memiliki antarmuka pengguna berbasis web yang disebut "Jenkins Dashboard" atau "Jenkins UI“.



[4] Jenkins Data

1) Jenkins tidak membutuhkan database untuk menyimpan seluruh konfigurasi dan datanya.
2) Jenkins akan secara otomatis membuat **folder .jenkins** di home directory sistem operasi kita.
3) Di folder tersebut lah, Jenkins akan menyimpan seluruh datanya.
4) Jika kita menghapus **folder .jenkins**, secara otomatis kita akan menghapus seluruh data Jenkins.

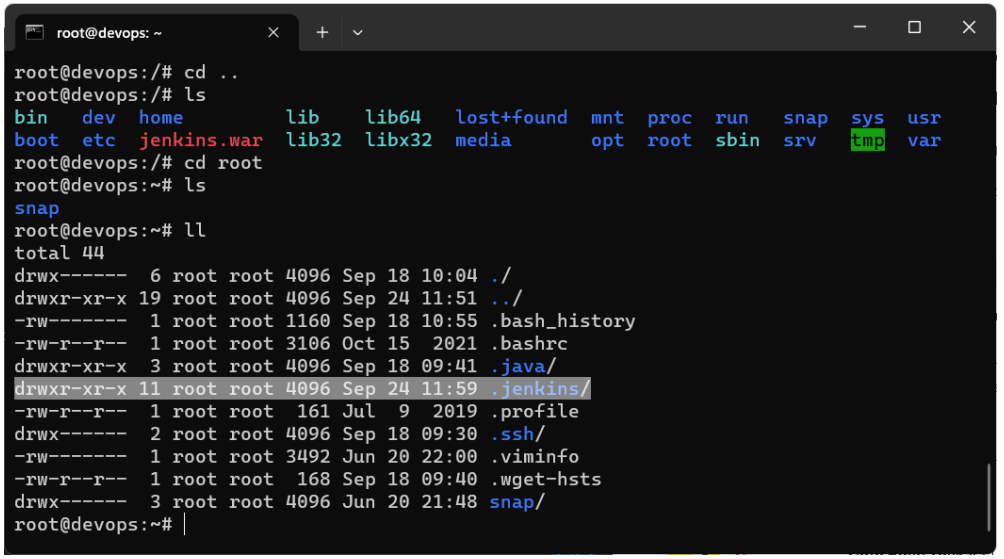
[1] Root akses:
- *sudo su*

[2] Cek Home Derectory di Ubuntu 22.04:
- *echo $HOME*

[3] Tampilkan forder (*long list*) di Ubuntu:
- *ll*

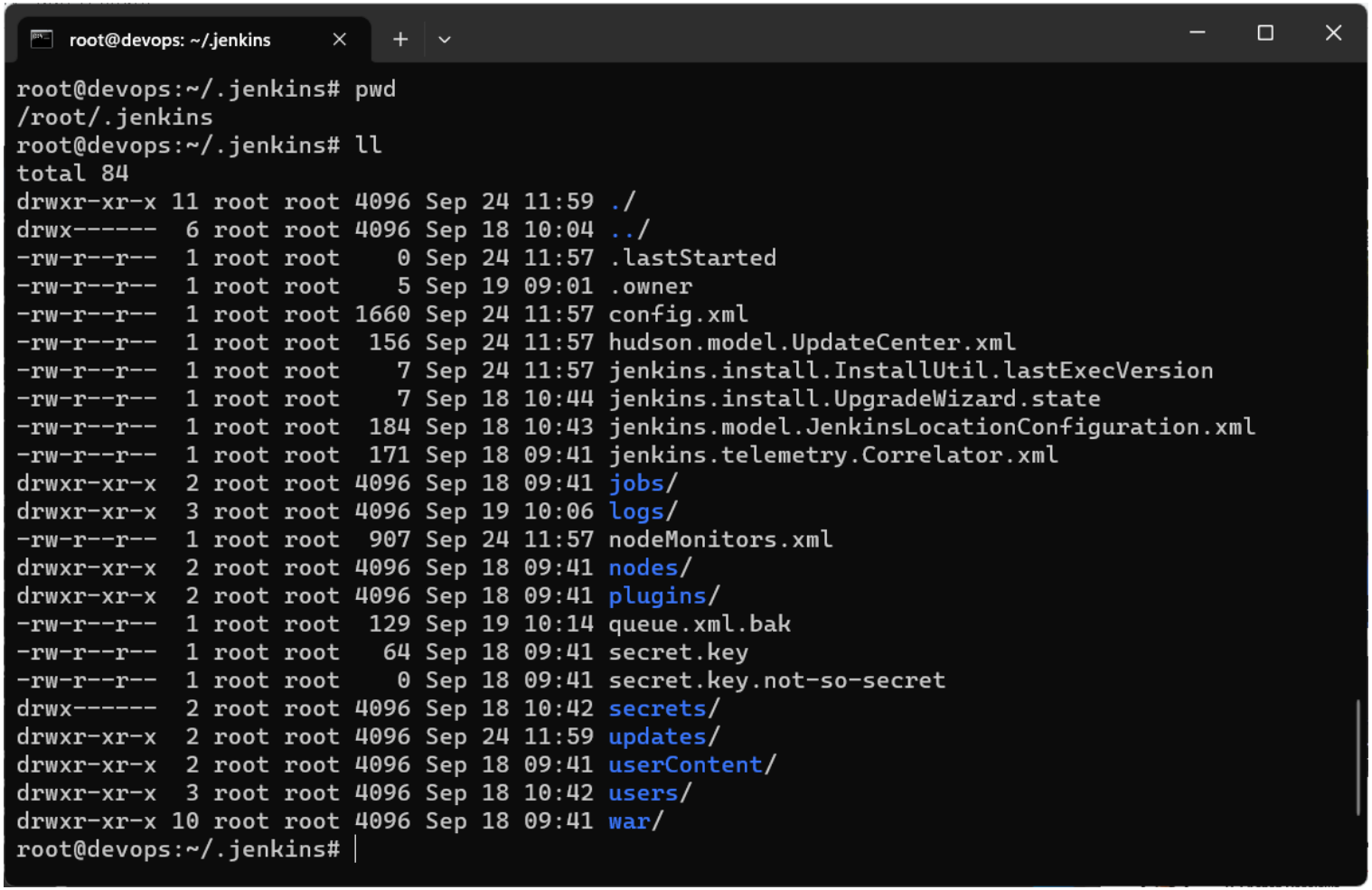
[4] Masuk di folder .jenkins:
- *cd .jenkins*

[5] Print Working Directory (PWD):
- *pwd*



Saya install Jenkins di *root*, makanya lokasinya ada di **root directory**, temukan lokasi folder *.jenkins* anda.

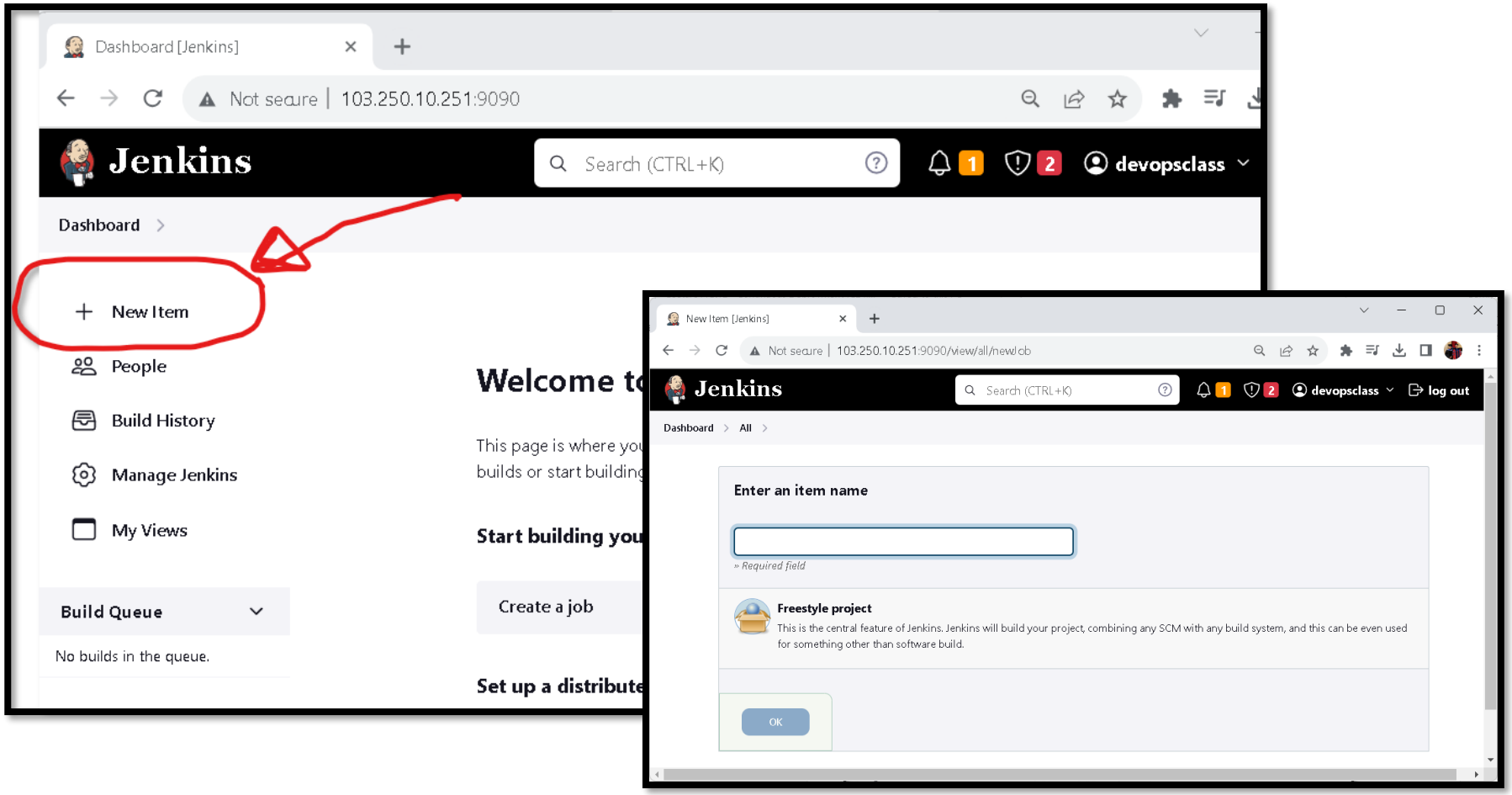
[5] Folder Backup/Restore ".jenkins"



Folder .jenkins ini dapat digunakan untuk folder backup di Jenkins.

[6] Create Job

1) Job adalah panggilan untuk **tugas automation (*project/task*)** di Jenkins.
2) Saat kita menginstall Jenkins, tidak ada satupun Job sama sekali.
3) Kita bisa membuat Job di Jenkins menggunakan web Jenkins.
4) Kita dapat meng-click di **button "New Item"** untuk memulai membuat Job baru.



Button "New Item" Jenkins

Saat mengklik button "New Item" di Jenkins, kita sedang mencoba membuat sebuah "Job“. Dalam konteks Jenkins, "Job" adalah sebuah tugas (*task*) atau pekerjaan (job/project) yang akan dijalankan oleh Jenkins secara otomatis. Job juga dapat diartikan berbagai jenis tugas, termasuk tetapi tidak terbatas pada:

1) **Job Integrasi Terus-Menerus (Continuous Integration):** Jenis *job* ini digunakan untuk mengotomatisasi proses mengambil kode dari repositori sumber, mengkompilasi, menjalankan pengujian otomatis, dan melaporkan hasilnya. Job CI digunakan untuk memastikan bahwa perubahan kode yang diajukan oleh pengembang bekerja dengan baik dengan kode yang ada.

2) **Job Pengiriman Terus-Menerus (Continuous Delivery atau Continuous Deployment):** Jenis *job* ini digunakan untuk mengotomatisasi proses pengiriman dan penyebaran perangkat lunak ke lingkungan produksi atau pengujian. Job CD digunakan untuk merilis *software* ke dalam lingkungan produksi secara terus-menerus atau berkala.

3) **Job Rutin:** Jenkins juga dapat digunakan untuk menjadwalkan pekerjaan rutin seperti **backup**, **penghapusan file tertentu**, **pemindaian keamanan (security scanning)**, dan tugas-tugas lainnya yang harus dijalankan secara berkala.

4) **Job Khusus:** Kita dapat membuat pekerjaan khusus sesuai dengan kebutuhan proyek. Ini mungkin termasuk **menjalankan skrip**, **memicu build manual**, m**engirim pemberitahuan**, dan tugas-tugas lainnya.

Saat kita membuat jenkins *job* baru, kita diberi berbagai opsi dan konfigurasi untuk menentukan bagaimana *job* tersebut akan dijalankan, apa yang akan dilakukan, dan bagaimana akan dijadwalkan atau diaktifkan.

[2]

Simple Automated Message using Jenkins

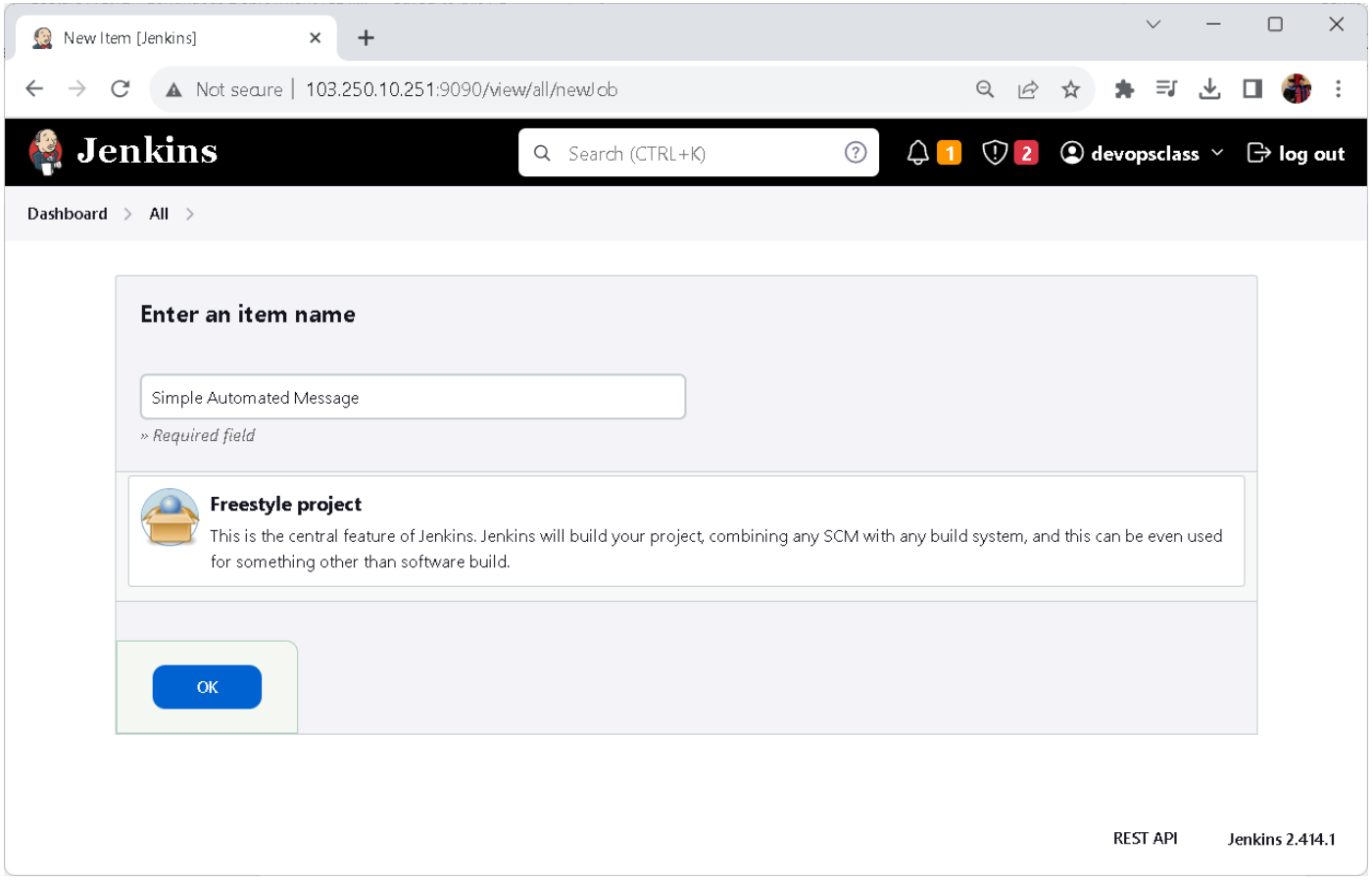
Jenkins

Simple Automated Message di Jenkins untuk perkenalan (basic CI/CD pipeline).

[1] Create Jenkins's Job

Kita akan membuat job yang hanya mencetak pesan ke konsol. Step awal yaitu membuat Job Jenkins:

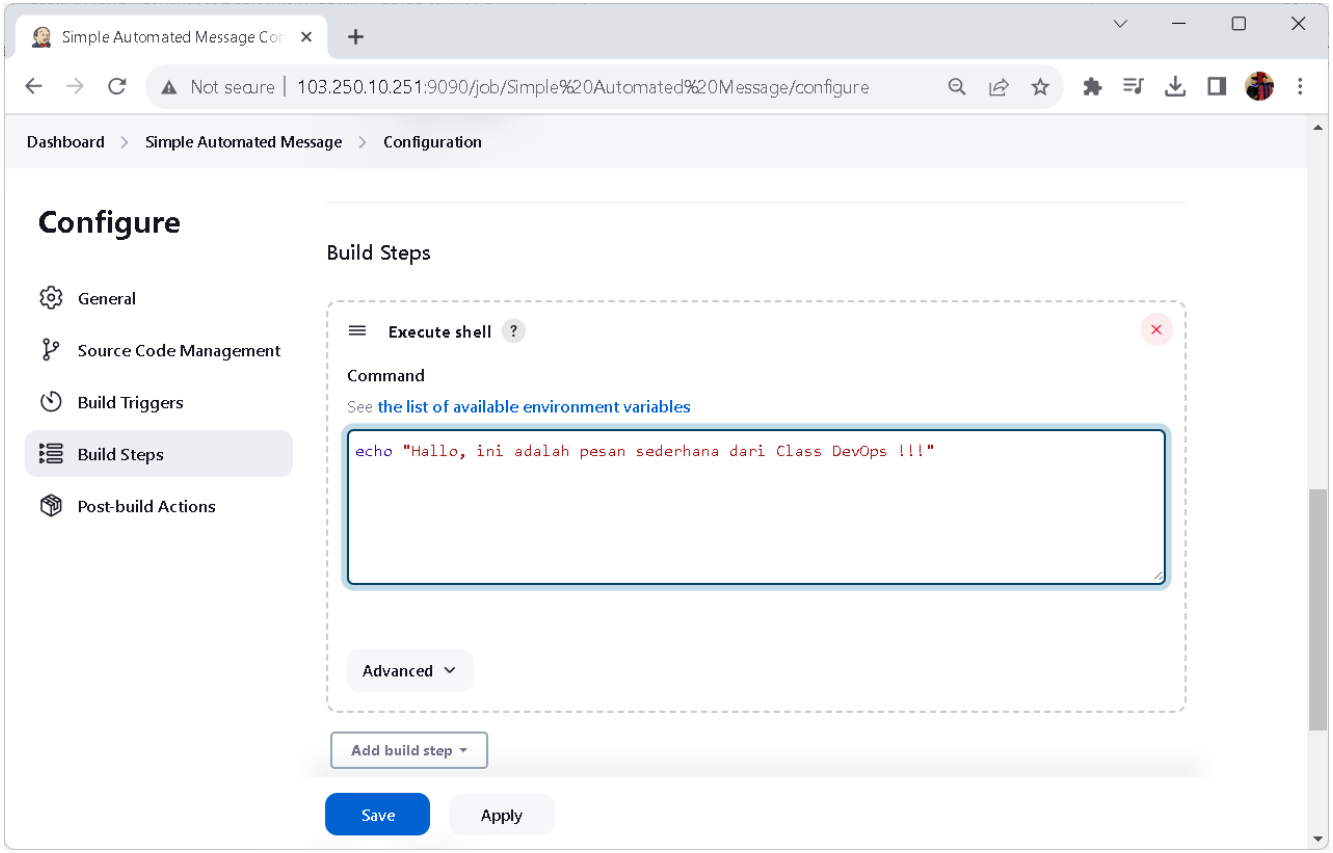
1) Buka dashboard Jenkins.
2) Klik "New Item" untuk membuat job baru.
3) Beri nama job (misalnya, "Simple Automated Message").
4) Pilih "Freestyle project" sebagai tipe proyek.



[2] Konfigurasi Job

Di halaman konfigurasi proyek Jenkins:

1) Di bawah bagian "**Build**", tambahkan sebuah "**Build step**" dengan memilih "**Execute shell**" (pilihan ini akan menjalankan perintah shell pada sistem operasi ubuntu).
2) Akan muncul kotak dan di dalam kotak teks yang tersedia, ketik perintah yang akan dicetak ke konsol. Misalnya, tulislah perintah seperti dibawah ini:
   echo "Hallo, ini adalah pesan sederhana dari Class DevOps !!! "

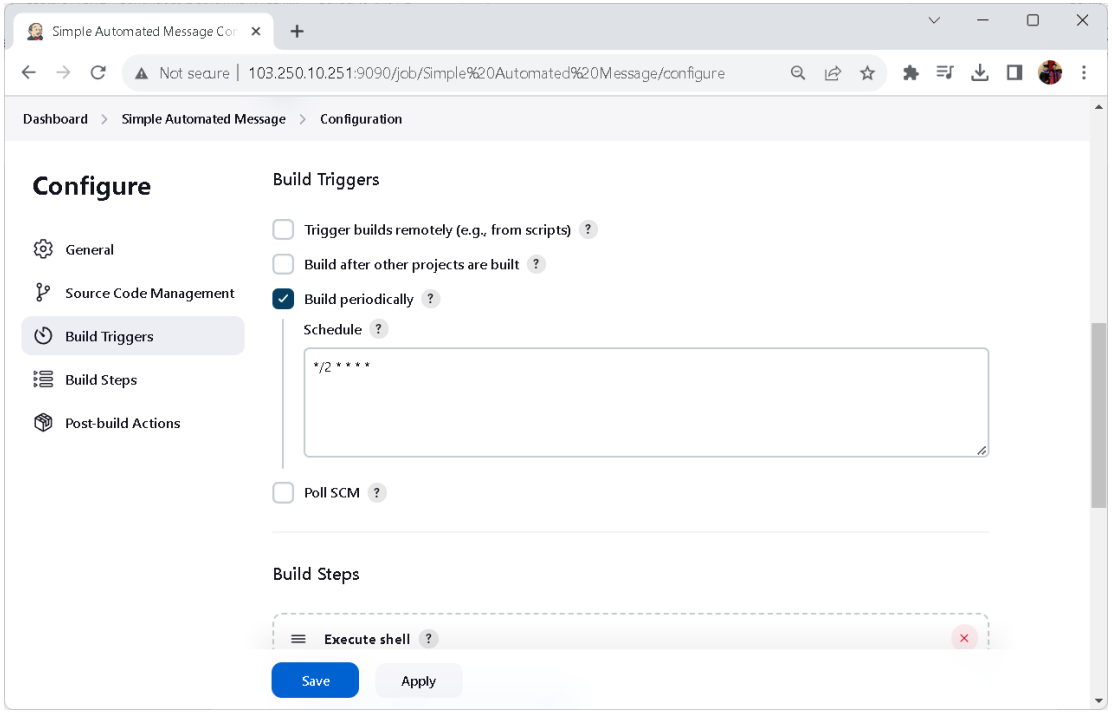


[3] Konfigurasi Job

1) Di halaman konfigurasi job, temukan bagian "**Build Triggers**".

2) Aktifkan opsi "**Build periodically**".

3) Di dalam kotak teks yang tersedia, masukkan **ekspresi cron** untuk menjadwalkan job setiap **2 menit**. **Ekspresi cron** untuk setiap 2 menit adalah ***/2 * * * ***.



Cron Expression Ubuntu 22.04

Ekspresi cron */2 * * * * memiliki arti sebagai berikut:

1) ***/2 :** Ini adalah bagian pertama dari ekspresi cron dan **mengatur "menit”**. Tanda asterisk (*) berarti "setiap," dan */2 berarti "setiap dua menit." Dengan kata lain, job akan dijadwalkan untuk dijalankan setiap dua menit pada menit yang berapapun.

2) * : Ini adalah bagian kedua dari ekspresi cron dan **mengatur "jam”.** Tanda asterisk (*) dalam bagian ini berarti "setiap jam," jadi job akan dijalankan setiap jam.

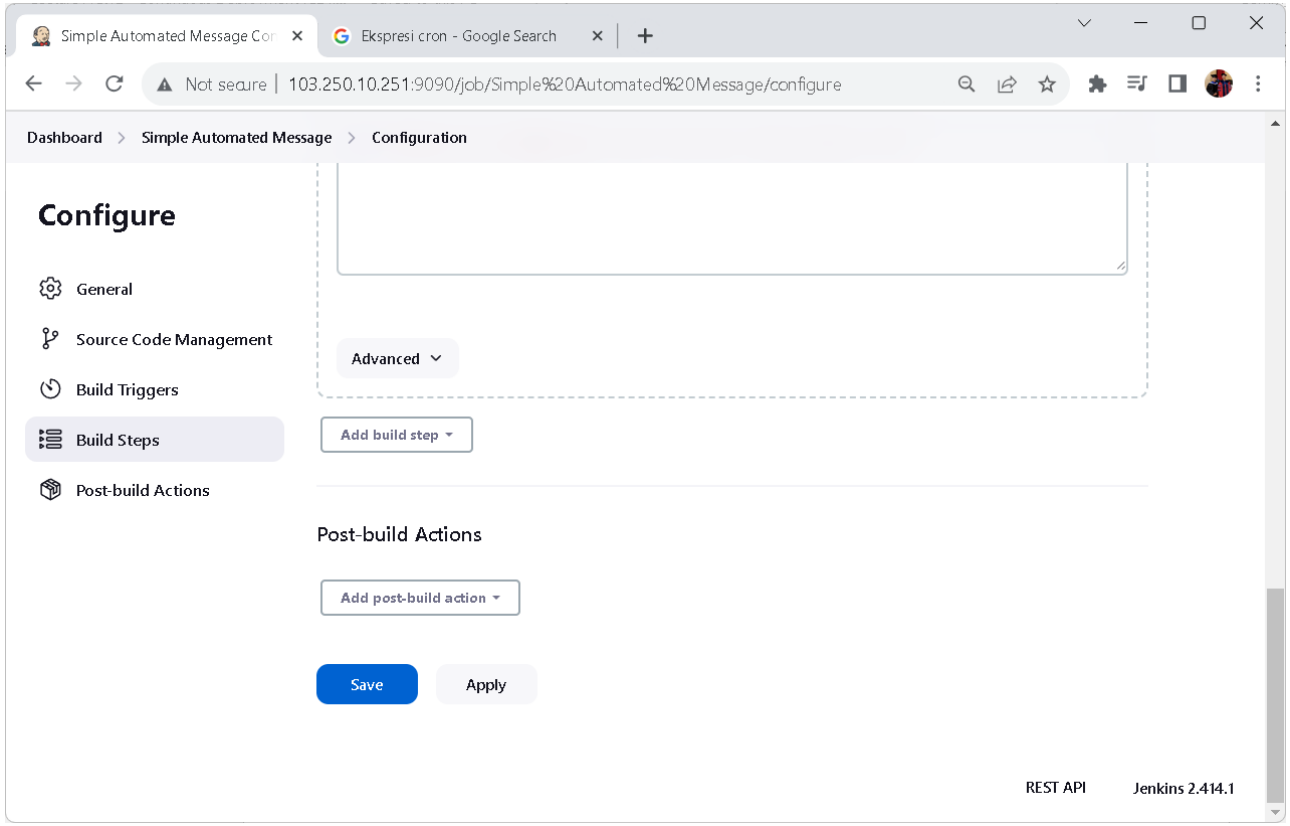
3) * : Ini adalah bagian ketiga dari ekspresi cron dan **mengatur "hari dalam bulan”.** Tanda asterisk (*) dalam bagian ini berarti "setiap hari dalam bulan," sehingga job akan dijadwalkan setiap hari dalam sebulan.

4) * : Ini adalah bagian keempat dari ekspresi cron dan **mengatur "bulan”.** Tanda asterisk (*) dalam bagian ini berarti "setiap bulan," jadi job akan dijalankan setiap bulan.

5) * : Ini adalah bagian kelima dari ekspresi cron dan **mengatur "hari dalam seminggu".** Tanda asterisk (*) dalam bagian ini berarti "setiap hari dalam seminggu," sehingga job akan dijalankan setiap hari dalam seminggu.

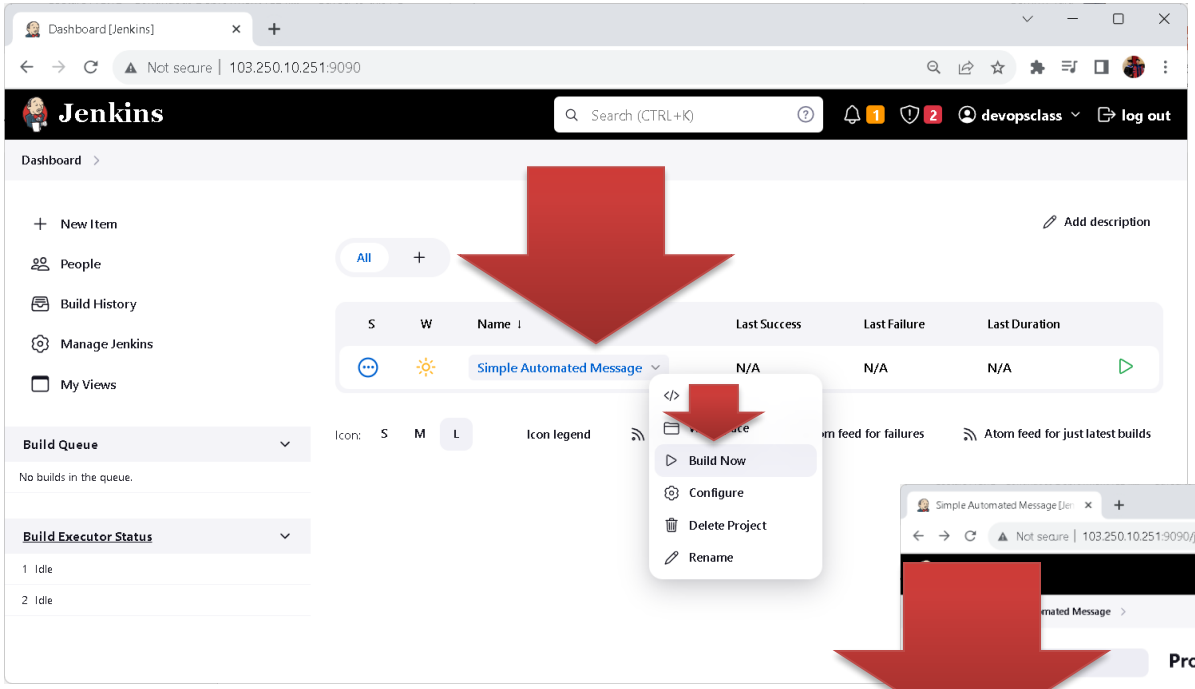
[4] Save Konfigurasi Jenkins

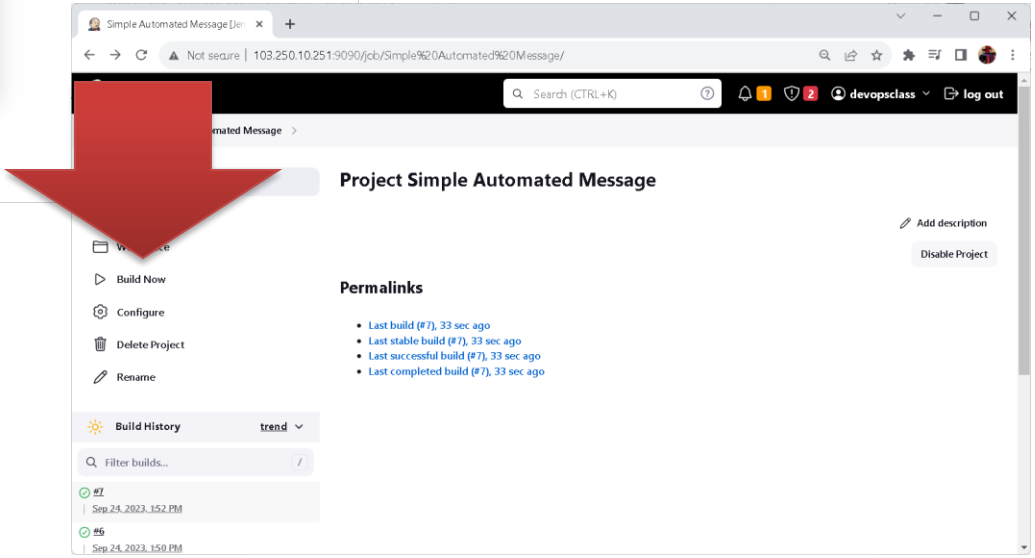
Klik "Save" untuk menyimpan konfigurasi *job Jenkins*.



[5] Run Job Jenkins

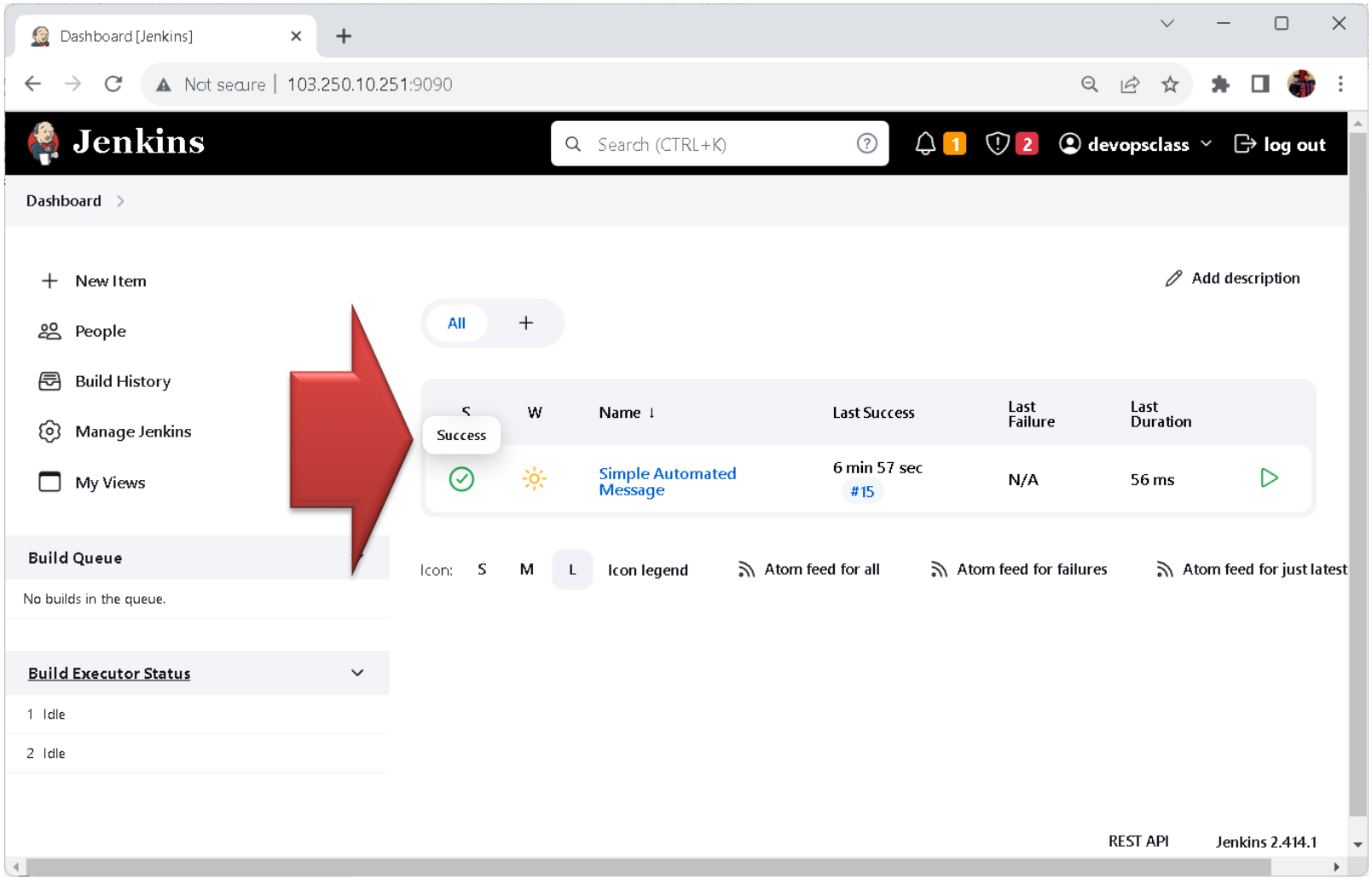
Kembali ke dashboard Jenkins dan temukan pekerjaan “**Simple Automated Message**" (atau sesuai nama yang diberikan). Klik pada job tersebut dan kemudian klik "**Build Now**" untuk menjalankannya.





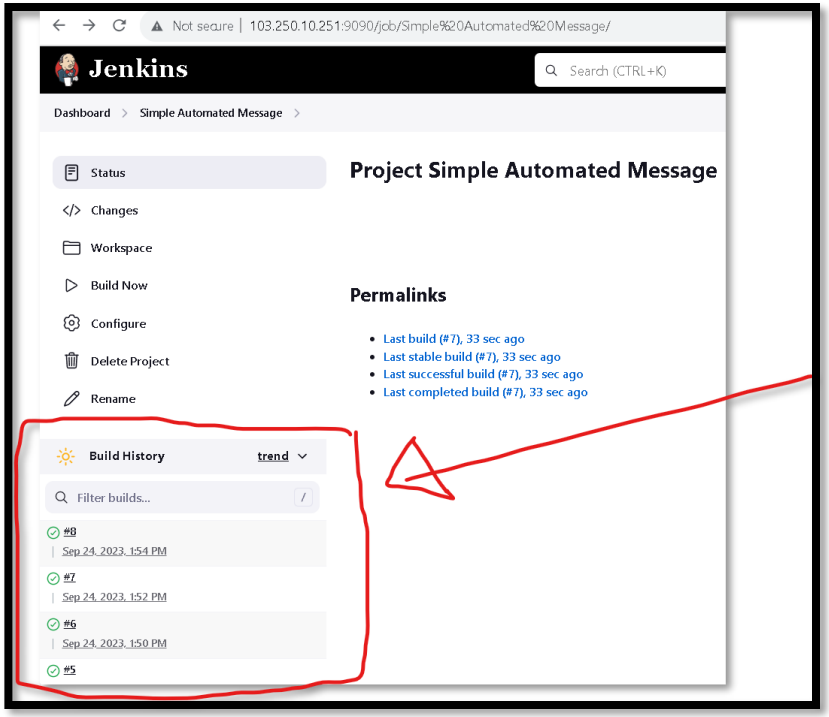
[6] Check Jenkins Status

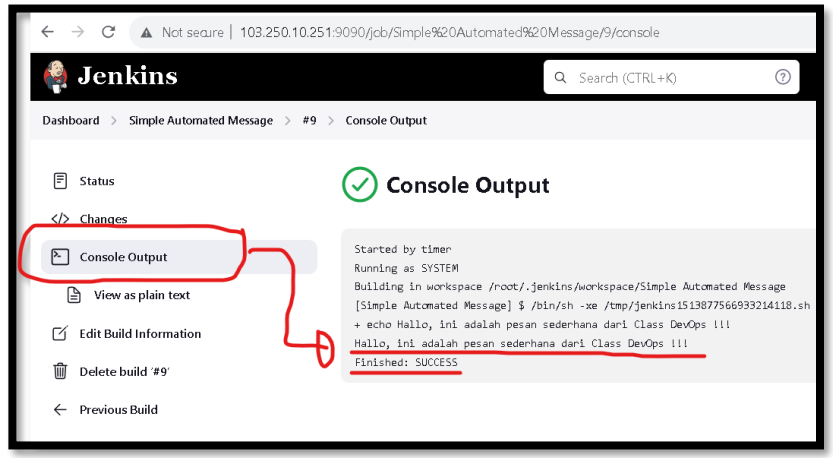
Kita dapat mengecek jenkins status pada **daftar job** di dashboard jenkins.



[6] View Jenkins Output

Setelah job selesai dijalankan, kita dapat melihat hasil dari pesan otomatis dengan mengklik job yang telah dibuat, click pada salah satu “**Build History**” dan melihat "**Console Output**" untuk melihat pesan yang dicetak ke konsol.

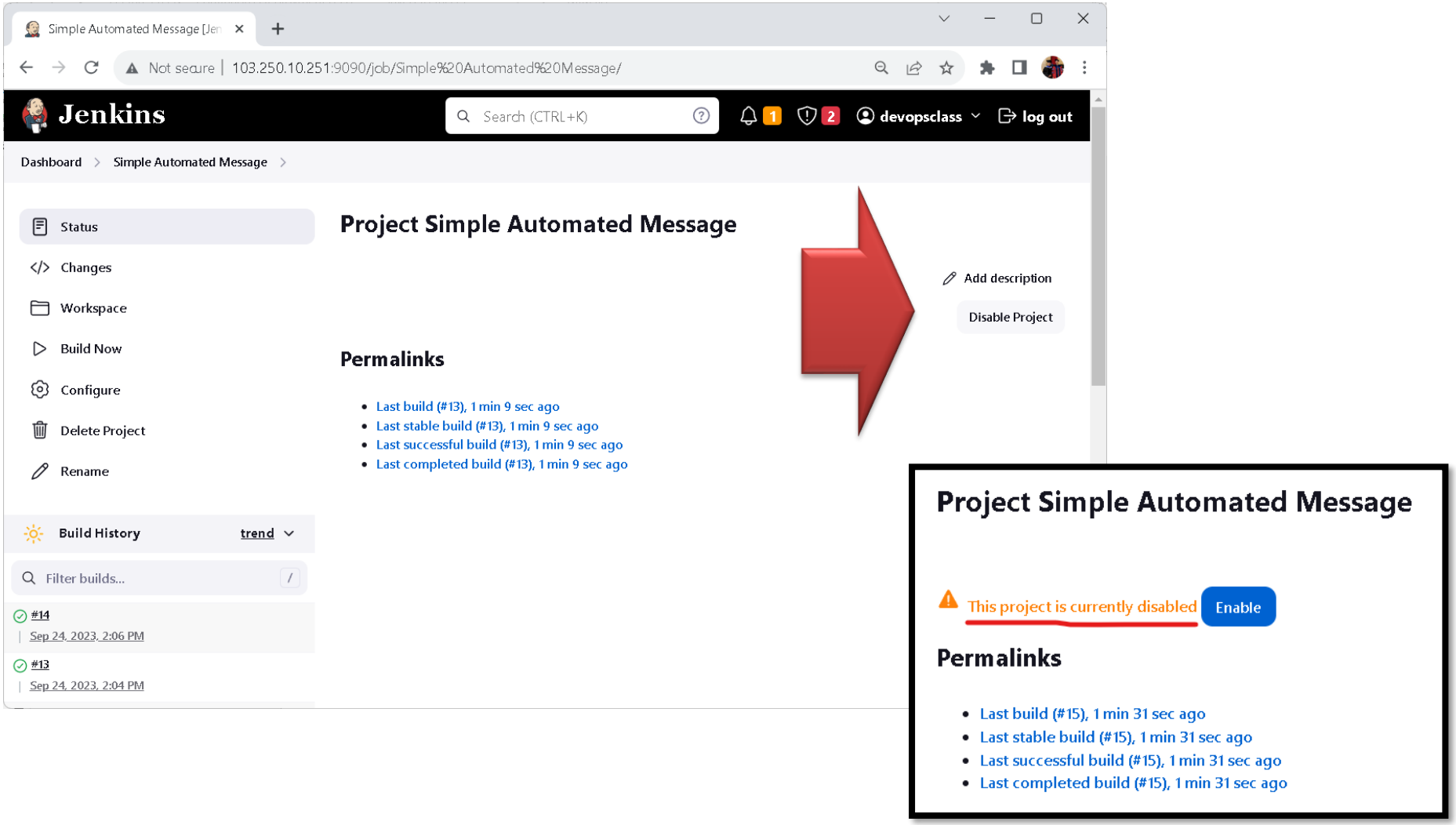




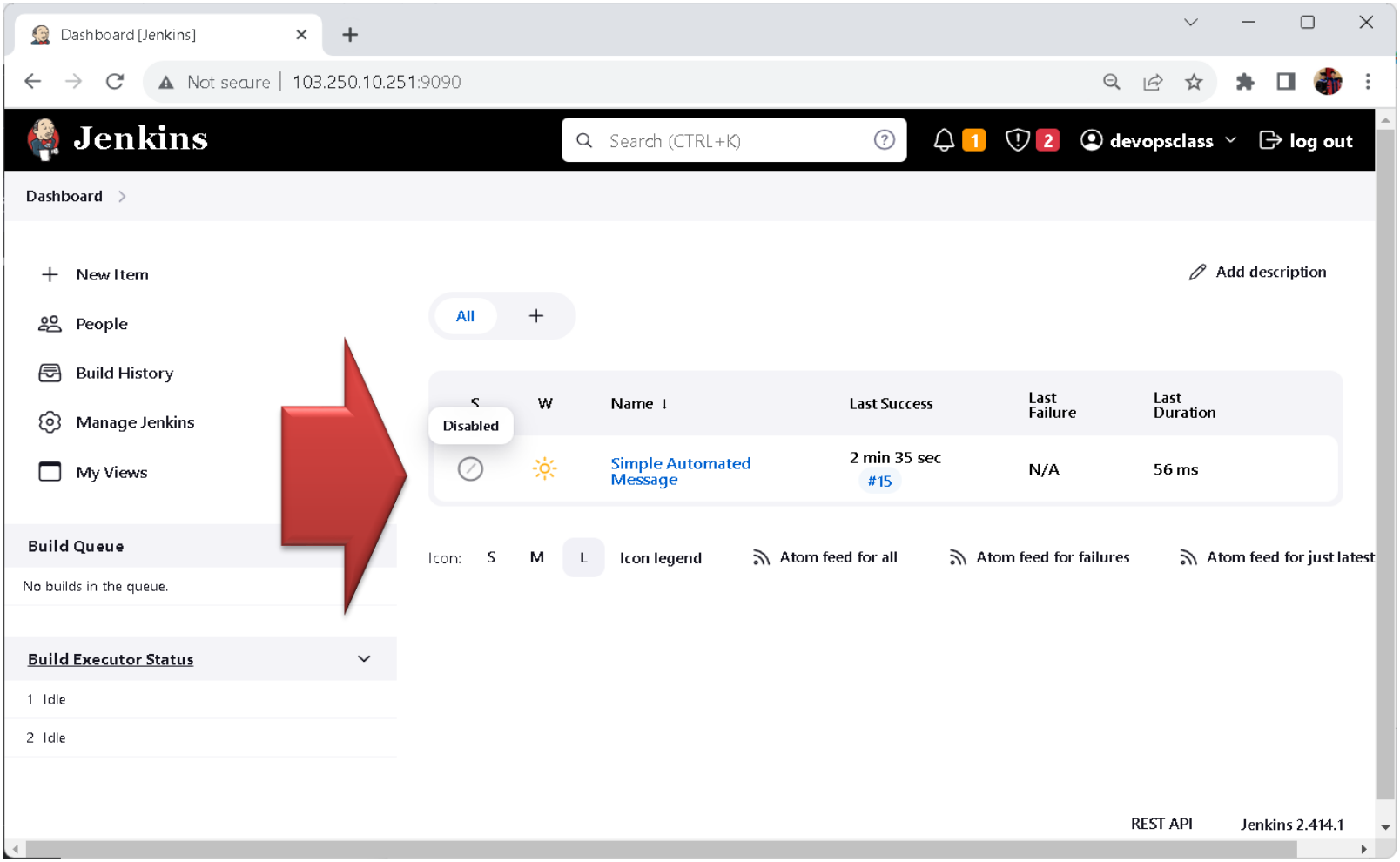
Perhatikan di Build History, historynya akan bertambah setiap 2 menit, karena ada perintah yang dieksekusi setiap 2 menit sekali.

[7] Dashboard Jenkins

Di halaman detail job, kita akan melihat opsi "**Stop**“, "**Abort**“ atau “**Disable Project**” (*tergantung pada versi Jenkins yang digunakan*) yang akan memungkinkan kita menghentikan job yang sedang berjalan.



[8] Check Jenkins Status



Next meeting, masing-masing kelompok akan di minta untuk membuat Job baru di Jenkins. Anggota kolompok yang akan diminta untuk jelaskan dan demo secara langsung di depan class.

[3]

Simple Automated BMKG Real-Time Earthquake API Monitoring with Jenkins

Jenkins

(Basic CI/CD pipeline)

Challenges

Automated Earthquake Monitoring using Jenkins

Tujuan Pembelajaran:

1) Students diberikan **TANTANGAN** untuk membuat Jenkins job otomatis mengakses/memanggil **Earthquake API** (misalnya data/api dari BMKG) dan memberikan laporan.

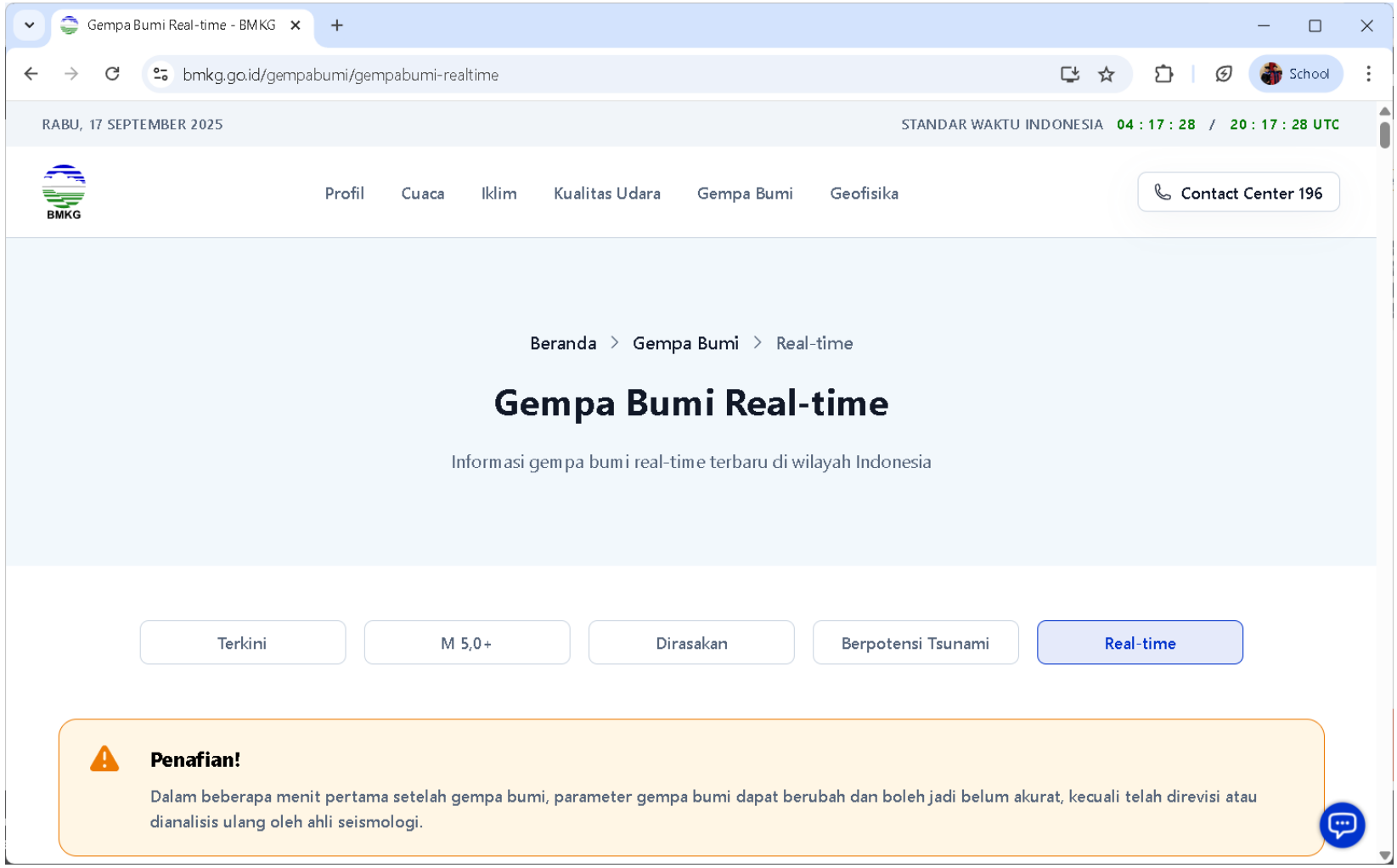
2) Setiap **1 menit sekali**, Jenkins mengambil data gempa untuk **beberapa daerah/negara yang berbeda** (tanpa kecuali, bebas untuk students dalam menentukan API gempa yang digunakan).

3) Hasil laporan ditampilkan di **Console Output** (~~opsional dapat disimpan dalam file atau bisa kirim ke email/telegram/whatsapp~~).

4) Masing-masing kelompok akan mempresentasikan hasil temuan ini dan dinilai secara langsung di dalam class.

BMKG – Gempa Bumi Real-time

https://www.bmkg.go.id/gempabumi/gempabumi-realtime



Endpoint BMKG

[1] Manual collect/Crowl data gempa:
https://www.bmkg.go.id/gempabumi/gempabumi-realtime

[2] Gempa terkini (M > 5 SR):
https://data.bmkg.go.id/DataMKG/TEWS/autogempa.json

[3] Gempa dirasakan (felt earthquake):
https://data.bmkg.go.id/DataMKG/TEWS/gempadirasakan.json

```
#!/bin/bash

# get page real-time gempa BMKG
URL="https://www.bmkg.go.id/gempabumi/gempabumi-realtime"

HTML=$(curl -s "$URL")

# parsing data / nama-kota di Sulut
echo "$HTML" | grep -i "Sulawesi Utara" | sed -n '1,5p'

# tambahkan juga magnitudo / lokasi / kedalaman
```

 # Izin eksekusi:
chmod +x bmkg_quake_monitor.sh

 # Jenkins Job
Buat **Freestyle Project** → Build Step → Execute shell:
./bmkg_quake_monitor.sh

```
#!/bin/bash
BASE_URL="https://data.bmkg.go.id/DataMKG/TEWS/autogempa.json"

echo "=== BMKG Earthquake Report $(date) ==="

# Ambil data JSON dari BMKG
RESPONSE=$(curl -s $BASE_URL)

# Parsing dengan jq
TANGGAL=$(echo $RESPONSE | jq -r '.Infogempa.gempa.Tanggal')
JAM=$(echo $RESPONSE | jq -r '.Infogempa.gempa.Jam')
KOORDINAT=$(echo $RESPONSE | jq -r '.Infogempa.gempa.Coordinates')
LOKASI=$(echo $RESPONSE | jq -r '.Infogempa.gempa.Wilayah')
MAG=$(echo $RESPONSE | jq -r '.Infogempa.gempa.Magnitude')
KEDALAMAN=$(echo $RESPONSE | jq -r '.Infogempa.gempa.Kedalaman')
DIRASAKAN=$(echo $RESPONSE | jq -r '.Infogempa.gempa.Dirasakan')

# Cetak laporan
echo "Tanggal    : $TANGGAL"
echo "Jam        : $JAM"
echo "Magnitudo  : $MAG SR"
echo "Lokasi     : $LOKASI"
echo "Koordinat  : $KOORDINAT"
echo "Kedalaman  : $KEDALAMAN"
echo "Dirasakan  : $DIRASAKAN"
```

Contoh Output di Jenkins Console

=== BMKG Earthquake Report Wed Sep 17 18:45:01 WIB 2025 ===

Tanggal : 17-Sep-2025
Jam : 18:40:12 WIB
Magnitudo : 5.3 SR
Lokasi : Maluku Tenggara
Koordinat : -5.12,132.45
Kedalaman : 10 km
Dirasakan : II-III Saumlaki

Untuk python code atau shell script bisa buat versi kalian sendiri per kelompok. Tidak ad step by step yang akan diberikan, setiap kelompok wajib untuk membuat tantangan ini karena akan di nilai.

<u>Opsional/tidak wajib untuk mahasiswa:</u>

- ✓ Simpan hasil ke earthquake.log (append per menit).
- ✓ Kirim notifikasi otomatis ke **Telegram / WhatsApp Gateway** jika ada gempa baru. Bisa gunakan Fonnte https://fonnte.com/.
- ✓ Visualisasi hasil monitoring di web dashboard (grafik/gmap).

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU